pointer

2024年11月5日 0:52

variable-addressable
on left->lvalue: location of the memory storage (read&write)
on write->rvalue: value in the storage(read only)
(x+10)=6; invalid表达式必须是可修改的左值
++x x should be pass by reference; modify +1; return new x by reference; returned x is an lvalue
x++ x should be pass by reference; save x in temporary local variable; modify+1; return old value of x in the local variable by value; returned value is rvalue

get the address of variable: &variable

pointer: type* variable
a pointer variable stores the address of another variable
y store x = y points to x
pointer variable is a variable which has its own address
get the content through pointer variable: *pointervariable
int

A pointer can point to
objects of basic types: char, short, int, long, oat, double, etc.
objects of user-de ned types: struct, class (discussed later) another pointer! ***
even to a function function pointer

const pointer: type* const variable=&another variable
const pointer must be initialized when it is defined!
const pointer once defined cannot be changed to point something else->content are free to change!

pointer to a const object: const type* variable
pointer direction is free to change but content cannot be changed when accessed through pointer!!->can still be changed by the object directly
initialization not necessary

pass by reference PBR = pass by value PBV + pointer
To simulate the e ect of PBR, one may pass the address of an object to a function
Inside the function, the object is represented by a pointer.
Then one may change the objects value by dereferencing the objects pointer inside the function

pointer for struct

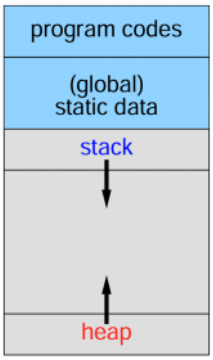
- Two ways to access struct members through a pointer:
  1. Dereference the pointer and use the . operator.

```
Point a;                // a contains garbage
Point* ap = &a;         // Now ap points to a

// Dereference ap, then use the . operator
(*ap).x = 3.5;
(*ap).y = 9.7;
```

  2. Directly use the → operator.

```
Point a;                // a contains garbage
Point* ap = &a;         // Now ap points to a

// No dereferencing when using the -> operator
ap->x = 3.5;
ap->y = 9.7;
```

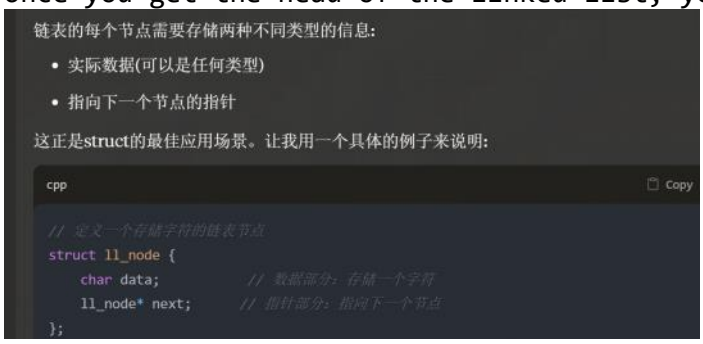
static objects
normal variables require static memory allocation. memory are allocated by the complier during compilation->**static objects**
when go out of their **scope--memory are released automatically back to RAM**(computer memory store)
dynamic objects
an object/array of objects whose memory is
---allocated at runtime explicitly by you using the operator new
---will persist even out of scope!
---has to be deallocated at runtime explicitly by you using the operator delete
*static objects are managed using a data structure called stack
*dynamic objects are managed using a data struc called heap



dynamic memory alllocation: type* pointer-vari-name=new type
eg. int* pointname = new int;
->allocate an amount of memory equal to sizeof(int) to program
new return a value which is the address of the starting location of the memory <-the memory is unnamed, can only be hold and accessed by pointer pointername.
!!pointer itself is static, the unnamed memories returned by the new operator is dynamic
delete: delete pointer-variable-name内存被释放，但pointer仍然储存原有地址->dangling pointer(a pointer that points to a location whose memory is deallocated)-when trying to dereference a dangling pointer, runtime error occurs(the memory is no longer accesible/the memory is reallocated to other functions / programs
so reset&initialize the pointer to nullptr!
Memory leak:allocated memory not longer needed but not released
too much memory leak->runtime error

linked list: dynamic, flexible size
object in linked list-**node**

The typical C++ definition for a node in a linked list is a struct (or later class)
The first and the last node of a linked list always need special attention.
For the last node, its next pointer is set to nullptr to tell that it is the end of the linked list.
We need a pointer variable, usually called head to point to the rst node.
Once you get the head of the linked list, you get the whole list!




```
ll_node* head = nullptr; // An empty linked list
```

head=xxx;
delete xxx;
xxx=nullptr;
插入/删除注意前后链接->prev->next 和 new_current->next
head要重点关照->
insert时如果插入第一位：new_node->next=head;head=new_node;
正常处理：new_node->next=prev->next;prev->next=new_node;
delete时如果删除第一位：head=current->next;delete current;(current=nullptr;)
(此时prev=nullptr;current=head)
正常处理：prev->next=current->next;delete current;
delete whole list by recursion:

```
#include "ll_cnode.h"   /* File: ll_delete_all.cpp */

// To delete the WHOLE linked list, given its head by recursion.
void ll_delete_all(ll_cnode*& head)
{
    if (head == nullptr) // An empty list; nothing to delete
        return;

    // STEP 1: First delete the remaining nodes
    ll_delete_all(head->next);

    // For debugging: this shows you what are deleting
    cout << "deleting " << head->data << endl;

    delete head;        // STEP 2: Then delete the current nodes
    head = nullptr;     // STEP 3: To play safe, reset head to nullptr
}
```

binary tree:any node of a binary tree has 2 sub-trees (children): left sub-tree (child) and right sub-tree (child).

```
struct btree_node       // A node in a binary tree
{
    int data;
    btree_node* left;   // Left sub-tree or called left child
    btree_node* right;  // Right sub-tree or called right child
};
```

array as pointer
a pointer variable supports 2 arithmetic operations+-

- If you have `<type> x; <type>* xp = &x;`, then
  - `xp + N` == &x + sizeof(<type>) × N.
  - `xp - N` == &x - sizeof(<type>) × N.
- The result of pointer arithmetic should be a valid address, otherwise, dereferencing it may lead to segmentation fault!

the array identifier can also be interpreted as a pointer to the first array element->can be treated as a const pointerS

- Any pointer pointing to an array can be used to access all elements of the array instead of the original array identifier.

```cpp
#include <iostream>     /* File: array-by-another-pointer.cpp */
using namespace std;

int main()
{
    int x[] = { 11, 22, 33, 44 };
    int* y = x; // Both y and x point to the 1st element of array

    // Modify the array through pointer y
    for (int j = 0; j < sizeof(x)/sizeof(int); ++j)
        y[j] += 100;

    // Print the array through pointer x
    for (int j = 0; j < sizeof(x)/sizeof(int); ++j)
        cout << x[j] << endl;
    return 0;
}
```

- Using pointer arithmetic, you may "move" a pointer to point to any array element.
- Dereferencing a pointer to an array element then obtains the element — and you can use it as either lvalue or rvalue.

Syntax: new a Dynamic Array

```
<type>* <pointer-variable> =
            new <type> [ <integer-expression> ] ;
```

Examples: Use of the new Operator

```cpp
int array_size; cin >> array_size; // Unknown till runtime
int* x = new int [array_size];
for (int j = 0; j < array_size; ++j)
    x[j] = j;  // Actually a pointer but treated like an array
```

```cpp
int arr[5];       // 静态数组,栈上分配
int* p = new int[5]; // 动态数组,堆上分配
```

x[i]等价于*(x+i)

删除内存：delete[] x;(x=nullptr;)

*boolalpha -> 输出从0/1变成false/true

```cpp
#include <iostream>     /* File: 2d-dynamic-array-functions.cpp */
using namespace std;

int** create_matrix(int num_rows, int num_columns) {
    int** x = new int* [num_rows];      // STEP 1
    for (int j = 0; j < num_rows; ++j) // STEP 2
        x[j] = new int [num_columns];
    return x;
}

void print_matrix(const int* const* x, int num_rows, int num_columns) {
    for (int j = 0; j < num_rows; ++j)
    {
        for (int k = 0; k < num_columns; ++k)
            cout << x[j][k] << '\t';
        cout << endl;
    }
}

void delete_matrix(const int*const * x, int num_rows, int num_columns) {
    for (int j = 0; j < num_rows; ++j) // Delete is done in reverse order
        delete [] x[j];                // (compared with its creation)
    delete [] x;
}
```

Notice that, numerically, we have

- x == &x[0] != &x[0][0]
  ⇒ x points to x[0] (and not x[0][0] as in static 2D array)
- &x[j] == x+j
  ⇒ a proof of the pointer arithmetic.
- x[j] == &x[j][0]
  ⇒ x[j] points to the first element of the *j*th row.

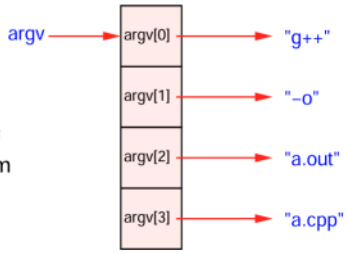
- 第一层：x (0x14ea5010)
  - x本身是一个指针，指向地址0x14ea5010
  - 在这个地址上存储的是第二层指针的地址(0x14ea5030)，也就是x[0]的地址
  - x和&x[0]是相同的值，因为它们都表示第二层指针的位置
- 第二层：x[0] (0x14ea5030)
  - x[0]是一个指针，位于地址0x14ea5030
  - 它存储的是实际整数数组的首地址
  - 这就是x[0][0]所在的位置
  - 从这里开始就是实际的整数数据了

```
x == &x[0]          // 都是 0x14ea5010
x[0] == &x[0][0]    // 都是 0x14ea5030
```

- Up to now, you write the main function header as int main( ) or int main(void).
- In fact, the general form of the main function allows variable number of arguments (overloaded function).

```
int main(int argc, char** argv)
int main(int argc, char* argv[ ])
```

- argc gives the actual number of arguments.
- argv is an array of char*, each pointing to a character string.
- e.g. g++ -o a.out a.cpp calls the main function of the g++ program with 3 additional commandline arguments. Thus, argc = 4, and



```cpp
#include <iostream> /* File: 2d-dynamic-array-main-with-argv.cpp */
using namespace std;
int** create_matrix(int, int);
void print_matrix(const int* const*, int, int);
void delete_matrix(int**, int, int);

int main(int argc, char** argv)
{
    if (argc != 3)
    { cerr << "Usage: " << argv[0] << " #rows #columns" << endl; return -1; }

    int num_rows = atoi(argv[1]);
    int num_columns = atoi(argv[2]);
    int** matrix = create_matrix(num_rows, num_columns);

    // Dynamic array elements can be accessed like static array elements
    for (int j = 0; j < num_rows; ++j)
        for (int k = 0; k < num_columns; ++k)
            matrix[j][k] = 10*(j+1) + (k+1);

    print_matrix(matrix, num_rows, num_columns);
    delete_matrix(matrix, num_rows, num_columns);
    matrix = nullptr; // Avoid dangling pointer
    return 0;
} /* g++ 2d-dynamic-array-main-with-argv.cpp 2d-dynamic-array-functions.cpp */
```

普通运行命令eg: ./lab3

此时运行命令eg. ./lab3 3 4(argc=3)